Encodage binaire des données

INF-335 AU2021

L'encodage de données consiste en transposer les données sous une forme binaire permettant de reproduire leur état initial ultérieurement. Pour y arriver, on va se créer un tableau de tous les octets composant notre donnée.

C'est une étape nécessaire à l'échange de données sur le réseau, ou tout simplement pour les écrire dans un fichier.

Lorsqu'on parle d'encodage, la première étape consiste à gérer les différents types primitifs. Les types complexes, tel les classes, ne sont, ultimement, qu'un agrégat organisé de types primitifs.

Encodage de types intégraux

Les différents types intégraux peuvent être trivialement encodé en isolant chacun des octets les composant.

Taille des types intégraux	
Type	Taille (en octets)
byte	1
short	2
int	4
long	8

L'aspect plus difficile à cette étape consiste à déterminer l'ordre d'apparition des octets. Deux méthodes s'offrent à nous :

1. Le petit boutisme (little endian)
2. Le gros boutisme (big endian)

Le petit boutisme consiste à faire apparaître l'octet de plus faible poids en premier, et de terminer avec l'octet de poids fort. Le gros boutisme est l'ordre inverse.

Les systèmes x86 travaillent par défaut en mode « **`petit boutisme`** ». L'ordre réseau standard est en « **`gros boutisme`** »

Pour isoler chacun des octets, on utilise les différents opérateurs binaires.

Exemple d'encodage d'un entier en petit boutisme :

```
int val = 0xBAADBEEF;
byte[] bytes = new byte[4];
bytes[0] = val &  0xFF;

// 0xBAADBEEF & 0xFF = 0xBAADBEEF & 0x000000FF = 0xEF

bytes[1] = (val >>> 8) & 0xFF;

// 0xBAADBEEF >>> 8 = 0x00BAADBE
// 0x00BAADBE & 0xFF = 0xBE

bytes[2] = (val >>> 16) & 0xFF;

// 0xBAADBEEF >>> 16 = 0x0000BAAD
// 0x0000BAAD & 0xFF = 0xAD

bytes[3] = (val >>> 24) & 0xFF;

// 0xBAADBEEF >>> 24 = 0x000000BA
// 0x000000BA & 0xFF = 0xBA

// bytes = [0xEF, 0xBE, 0xAD, 0xBA];
```

Encodage de booléens

Une variable booléenne a 2 états possibles. Il est donc possible, dans certains cas, d'encoder cette information sur 1 seul bit. Cela n'est cependant pas toujours possible, ni même souhaitable. Généralement, pour simplifier les opérations faites au moment de la lecture et de l'écriture, on essaie de travailler sur des octets entiers.

En fait, aucun API système ne permet de lire un seul bit à la fois – généralement votre système va lire un bloc de plusieurs octets d'un coup, et vous donner accès à un octet à la fois au minimum.

Pour cette raison, le cas minimaliste typique va consister à encoder le booléen sur 1 octet complet. Une valeur de 1 représenterait « true », et une valeur de 0 représenterait « false ».

```
boolean valTrue = true;
boolean valFalse = false;
byte[] encodeTrue = {valTrue ? 1 : 0}; // 1
byte[] encodeFalse = {valTrue ? 1 : 0}; // 0
```

On peut aussi, si la simplicité ou les performances le nécessite, l'encoder comme un entier de taille native (**`int`** sur système 32 bits, **`long`** sur système 64 bits)

Il est néanmoins possible, si l'organisation de nos données nous le permet, d'encoder plusieurs booléens dans un seul octet.

```
public static int setBitfield(int base, boolean val, int index) {
      int mask = 1 << index;
      return (base & ~mask) | ((val ? 1 : 0) << index);
}

public byte[] encodeInt(int v) {
      byte[] bytes {
            (byte)(v & 0xFF),
            (byte)((v >> 8) & 0xFF),
            (byte)((v >> 16) & 0xFF),
            (byte)((v >> 24) & 0xFF)
      } ;
      return bytes ;
}

public static void main(String… args) {
      int bitfield = 0 ;

      bitfield = setBitfield(bitfield, true, 0)
      bitfield = setBitfield(bitfield, true, 1)
      bitfield = setBitfield(bitfield, true, 4)

      // bitfield == 0x13 ; -- 0000 0000 0000 0000 0000 0000 0001 0011

      byte[] bytes = encodeInt(bitfield) ;

      //…

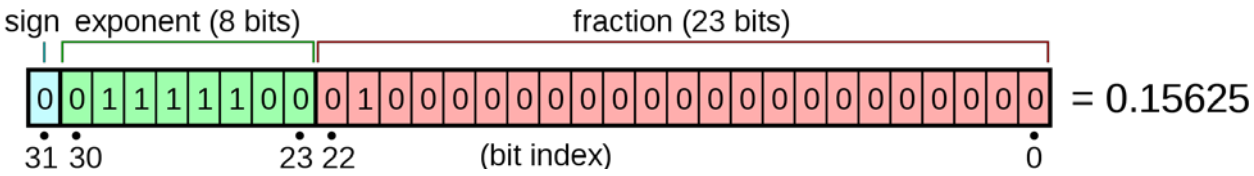
}
```

Encodage de nombres à virgules flottantes

Les nombres à virgules flottantes ont une forme assez particulière. Le bit de poids fort sert à déterminer le signe. S'en suit une certaine quantité de bits qui représentent un exposant, et le restant des bits étant nommé « mantissa » représentent la base du nombre.

Pour obtenir la valeur décimale du nombre, on utilise la formule :

$$(1.mantissa)_2 * 2 \text{ ^ } exposant$$



Dans cet exemple, nous aurions donc la formule :

$$1.01_b * 2^{\wedge(124-127)}{}_{10} = 1.25_{10} * 2^{\wedge(-3)}{}_{10} = 0.15625$$

En détail :

$(1.mantissa)_2$ est une valeur en binaire. Le mantissa est l'ensemble des bits tels qu'ils apparaissent dans le nombre.

Dans notre cas, nous avons, *1.01000…* ce qui se résume à *1.01*.

La forme décimale de ce nombre est :

$$(1 * 2^0) + (0 * 2^{-1}) + (1 * 2^{-2}) = 1.25_{10}$$

L'exposant, en tant que tel, est décalé de *127*. C'est ainsi que les exposants négatifs sont couverts. Il faut donc soustraire *127* de la valeur de notre exposant afin d'avoir sa valeur réelle.

$$01111100_2 = 124_{10}$$

$$124_{10} - 127_{10} = -3_{10}$$

Il en convient donc que les opérateurs binaires typiques auraient un effet substantiel et complexe sur la valeur des nombres à virgule flottante. C'est pour cette raison que le langage Java ne supporte pas ces opérations sur les **`floats`** ni les **`doubles`**.

La classe « **`Float`** » nous fournit donc des méthodes statiques (**`floatToIntBits`**, et **`intBitsToFloat`**) permettant de convertir un **`float`** en entier ayant comme structure binaire la représentation du **`float`** lui étant passé en paramètre, et de convertir un entier obtenu ainsi en **`float`**. Il suffit par la suite d'encoder cet entier.

Il en est de même pour la classe **`Double`** et les **`doubles`** avec les fonctions **`doubleToLongBits`** et **`longBitsToDouble`**.

```
float v = 0.15625f; //  0011 1110 0010 0000 0000 0000 0000 0000
int bits = Float.floatToIntBits(v);
int targetBits = 0x3E200000; //  0011 1110 0010 0000 0000 0000 0000 0000
System.out.println(bits == targetBits); // true
```

Encodage de texte

L'encodage de texte est nécessité de faire correspondre le glyphe à encoder (le char) à une séquence d'octet le représentant. La table ASCII est une méthode primitive d'encoder un ensemble limité de char (127) à l'intérieur de 7 bits.

La table ASCII supporte bien les différents glyphes de langues anglaises, ainsi qu'une certaine quantité de symboles de contrôles, de ponctuation et d'espacement.

La plupart des langues autres que l'anglais nécessite l'utilisation d'une quantité de glyphes dépassant les 127 inclus dans la table ASCII standard.

Par exemple, la langue japonaise va s'écrire à l'aide de 3 alphabètes différents utilisés dans différents contextes : les hiragana (46 charactères standard, un total de 114 avec les formes modifiés), les katakanas (48 charactères distincts), et les kanjis (plus de 50 000 kanjis).

La table ASCII est largement insuffisante pour encoder les textes japonais. Il en est de même pour plusieurs autres groupes linguistiques, tel les langues slaves (alphabète cyrillique), les autres langues asiatique (le vietnamien, le koréen, l'ourdou, …), les langues moyenne-orientales (l'arabe, le persan, …).

Le standard Unicode propose une association entre un code numérique unique et chacun des différents glyphes utilisés par différentes langues.

Pour encoder l'ensemble de ces glyphes, plusieurs encodage différents s'offrent à nous. Les encodages **`UTF-8`**, **`UTF-16`**, et **`UTF-32`** en sont les plus communs.

UTF-32 fonctionne en encodant le glyphe sur un entier de 32 bits. Chaque charactère occupe donc toujours 32 bits d'espace. Comparé aux autres encodages de texte, UTF-32 est le moins compact. En contrepartie, il est le plus simple, utilisant toujours une taille fixe pour chacun des charactères.

UTF-8 et UTF-16 fonctionnent tous les deux de la même façon, la différence étant la quantité d'espace minimale utilisée pour un glyphe. UTF-8 va travailler sur un octet (8 bits) à la fois, tandis que UTF-16 va travailler sur 2 octets (16 bits) à la fois.

Ces encodages fonctionnent en lisant un « code point » à la fois. Dépendamment de la valeur de ce code point, d'autres seront utilisés afin d'encoder entièrement sa valeur.

Exemple de principe d'encodage UTF-8			
Caractères codés	**Représentation binaire UTF-8**	**Premier octet valide (hexadécimal)**	**Signification**
U+0000 à U+007F	`0bbb·bbbb`	00 à 7F	1 octet, jusqu'à 7 bits
U+0080 à U+07FF	`110b·bbbb 10bb·bbbb`	C2 à DF	2 octets, jusqu'à 11 bits
U+0800 à U+FFFF	`1110·bbbb 10bb·bbbb 10bb·bbbb`	E0 à EF	3 octets, jusqu'à 16 bits

U+10000 à U+10FFFF	**1111**·**0**0*bb* 10*bb*·*bbbb* 10*bb*·*bbbb* 10*bb*·*bbbb*	F0 à F3	4 octets, jusqu'à 21 bits
	1111·**0**100 1000·*bbbb* 10*bb*·*bbbb* 10*bb*·*bbbb*	F4	

D'ordre générale, il est préféré d'utilisé **UTF-8** à **UTF-16**.

En Java, pour encoder une string, on va devoir spécifier l'encodage utilisé. Les encodages sont tenus dans la classe **java.nio.charset.Charset**. Vous pouvez obtenir un charset préexistant en allant chercher les définitions statiques dans la classe **java.nio.charset.StandardCharsets**.

Pour encoder une String :

```
String data = "toto toto";
byte[] bytes = data.getBytes(StandardCharsets.UTF_8);
```

Pour décoder une String :

```
String val = new String(bytes, StandardCharsets.UTF_8);
```

Notez bien que cette approche va uniquement encoder les charactères de la string sous un encodage particulier. Si nous écrivons ces données dans un fichier texte, il n'y a pas nécessairement de problème. Cependant, si nous écrivons ces données dans un fichier binaire, il se pourrait qu'on doive y ajouter des informations tel la quantité d'octets associés à cette String.

Encodage de types complexes

Encoder un type complexe, tel une classe, dans sa forme la plus simple, consiste à simplement encoder l'ensemble des attributs qui le composent. Le décodage va ainsi être de simplement décoder chacun de ces attributs.

```
public class ClasseSimple {

    private int a;
    private int b;
    private boolean c;

    public byte[] encode() {
        byte[] encodedA = encodeInt(a);
        byte[] encodedB = encodeInt(b);
        byte[] encodedC = encodeBoolean(c);

        return concat(encodedA, encodedB, encodedC);
    }
```

```
    public static byte[] encodeInt(int a) {
        byte[] bytes = {
                (byte)(a & 0xFF), (byte)((a >>> 8) & 0xFF),
                (byte)((a >>> 16) & 0xFF), (byte)((a >>> 24) & 0xFF)
        };
        return bytes;
    }

    public static byte[] encodeBoolean(boolean a) {
        return encodeInt( a ? 1 : 0);
    }

    public static byte[] concat(byte[]... arrs) {
        int resultIdx = 0;
        int resultLength = 0;

        for(byte[] arr : arrs) {
            resultLength += arrs.length;
        }

        byte[] result = new byte[resultLength];

        for(byte[] arr : arrs) {
            for(int i = 0; i < arr.length; i++, resultIdx++) {
                result[resultIdx] = arr[i];
            }
        }

        return result;
    }
}
```

Cette approche présente un inconvénient majeur découlant du fait qu'en temps normal, votre application va continuer à évoluer. Ainsi, les classes, auparavant simples, vont se faire ajouter des nouveaux champs qui devront être encodé aussi. Les fichiers binaires, une fois sauvegarder, ne changeront pas.

Il y a donc deux problèmes qui se présentent :

1. Le premier, on ouvre une ancienne version du fichier sur une nouvelle version de l'application. Des attributs de votre peuvent ne pas figurer dans votre fichier binaire.
2. Le second, on ouvre une nouvelle version du fichier sur une ancienne version de l'application. Le fichier binaire peut donc contenir des attributs que notre application ne connait pas.

Dans tous les cas, il faut supporter la possibilité que la structure du fichier lu ne corresponde pas à nos attentes.

Une stratégie commune d'encodage appelé le « **TLV** » adresse justement ce problème. La stratégie consiste à encoder un identifiant de « type » de ce que l'on encode, suivit de la longueur de l'attribut, et finalement de la valeur en soit.

L'identifiant de type sert à identifier ce que l'on encode en soit – quel attribut est encodé. Dans certains protocoles, c'est uniquement le type de donnée qui va être indiqué ici. Dans d'autres protocoles, c'est une étiquette (tag) identifiant l'attribut en soit. Cette information permet d'identifier exactement ce que l'on est en train de lire. Cela permet d'identifier si on reconnait l'information ou pas. Cela permet aussi de changer l'ordre d'encodage des champs – ce qui peut nous donner une flexibilité supplémentaire bien appréciée lorsque nos données sont très dynamiques.

L'information de longueur permet de sauter la valeur si elle n'est pas reconnue. Elle peut aussi nous faciliter la vie en ce qui concerne la quantité d'information à lire lorsqu'il est question de champs à taille variable. Par exemple, dans le cas d'un fichier binaire contenant une **`String`**, la taille de la **`String`** ne serait pas nécessairement toujours la même. Donc même si l'on connait l'attribut, il nous faut la taille pour savoir combien d'octets lire.

Le code qui suit illustre le genre de problème pouvant survenir lorsque l'on tente de lire une **`String`** encodée sans information de taille. Notez que la String **`a`** peut prendre n'importe quelle valeur.

```
public class ClasseAvecString {

    String a;
    public ClasseAvecString(String a) {this.a = a;}

    public byte[] encodeSansTaille() {
        // La taille n'est pas encodee, on retourne simplement l'ensemble des
octets
        return a.getBytes(StandardCharsets.UTF_8);
    }

    public static ClasseAvecString decodeSansTaille(InputStream is) throws
IOException {
        int len = 0; // Quelle quantite d'octet lit-on?
        byte[] data = is.readNBytes(len);
        return new ClasseAvecString(new String(data,
StandardCharsets.UTF_8));
    }
```

```
    public byte[] encodeAvecTaille() {
        byte[] stringData = a.getBytes(StandardCharsets.UTF_8);
        int sz = stringData.length;

        return concat(encodeInt(sz), stringData);
    }

    public static ClasseAvecString decodeAvecTaille(InputStream is) throws
IOException {
        // La taille est encode! on sait combien d'octets lire!
        int len = decodeInt(is.readNBytes(4));
        byte[] data = is.readNBytes(len);
        return new ClasseAvecString(new String(data,
StandardCharsets.UTF_8));
    }
}
```

Le code qui suit est une implémentation partielle et simplifiée d'encodage et de décodage avec TLV.

Notez que :

1. Les fonctions d'écriture et de lecture ne sont pas implémentez. On garde ça simple pour l'exemple.
2. Il pourrait être préférable de faire en sorte que ce soit la classe parent qui d'écrive l'étiquette et la taille d'**`ExempleTlv`**.
3. Présentement l'encodage se fait en retournant un tableau de bytes, et le décodage se fait à partir d'un **`InputStream`**. En temps normal, il serait préférable de normaliser l'API pour toujours utiliser un **`Stream`** (autant pour l'encodage que le décodage), ou toujours utiliser un tableau.
4. Compte tenu des points 2 et 3, on pourrait vouloir ajouter des méthodes permettant de simplement calculer la taille occupée par l'encodage de notre structure.

```
public class ExempleTlv {

    private enum Tag{
        TAG_EXEMPLE_TLV_A(0x13370A01),
        TAG_EXEMPLE_TLV_B(0x13370B01),
        TAG_EXEMPLE_TLV_C(0x13370A02),
        TAG_EXEMPLE_TLV(0x13370000);

        final public int value;
        private Tag(int v) {this.value = v;}
        public static Tag fromInt(int v) {
            for(Tag t : Tag.values()) {
                if (t.value == v) return t;
```

```
            }
            return null;
        }
    };

    private int a;
    private String b;
    private int c;

    public byte[] encode() {
        byte[] encodeA = encodeWithTagAndLength(Tag.TAG_EXEMPLE_TLV_A, a);
        byte[] encodeB = encodeWithTagAndLength(Tag.TAG_EXEMPLE_TLV_B, b);
        byte[] encodeC = encodeWithTagAndLength(Tag.TAG_EXEMPLE_TLV_C, c);

        int size = encodeA.length + encodeB.length + encodeC.length;

        return concat(encodeInt(Tag.TAG_EXEMPLE_TLV.value), encodeInt(size),
                encodeA, encodeB, encodeC);
    }

    public static byte[] encodeWithTagAndLength(Tag tag, int v) {
        return concat(encodeInt(tag.value), encodeInt(4), encodeInt(v));
    }

    public static byte[] encodeWithTagAndLength(Tag tag, String v) {
        byte[] stringData = v.getBytes(StandardCharsets.UTF_8);
        return concat(encodeInt(tag.value), encodeInt(stringData.length),
stringData);
    }

    public static ExempleTlv decode(InputStream is) throws IOException {
        if(Tag.fromInt(readInt(is)) != Tag.TAG_EXEMPLE_TLV) throw new
IOException();
        int size = readInt(is);
        int sizeRead = 0;

        ExempleTlv ex = new ExempleTlv();

        while(sizeRead < size) {
            Tag fieldTag = Tag.fromInt(readInt(is));
            int fieldSize = readInt(is);

            switch(fieldTag){
                case TAG_EXEMPLE_TLV_A:
                    ex.a = readInt(is);
                    break;
                case TAG_EXEMPLE_TLV_B:
                    ex.b = readString(is);
                    break;
                case TAG_EXEMPLE_TLV_C:
```

```
                    ex.c = readInt(is);
                    break;
                default:
                    is.skip(fieldSize);
            }

            sizeRead += size;
        }

        return ex;
    }
}
```